

EveryFinance/smart-contracts- Stellar@82ece6a36d544ac5ae5f053f6cca52e804d9b907

Status: COMPLETE
Report Generated: 5/8/2026, 5:27:57 AM
Scan Date: May 8, 2026, 5:21 AM

Findings Summary

Severity	Count
● Critical	1
● High	3
● Medium	7
● Low	9
● Info	0
Total	20

contracts/integration_tests/src/common.rs

● LOW Liquidity withdrawal does not burn share tokens

Description:

`MockPhoenixPool::withdraw\_liquidity` never burns/transfers `share\_amount` of the share token from `recipient`. A user can call `withdraw\_liquidity` repeatedly with the same shares (or even without holding shares, since no balance check exists here), receiving minted underlying tokens each time while reserves are decremented. In a real pool this would be a critical drain; here it can invalidate any test that assumes withdrawals are constrained by share ownership.

Affected Code:

```
pub fn withdraw_liquidity(
    env: Env,
    recipient: Address,
    share_amount: i128,
    min_a: i128,
    min_b: i128,
    _deadline: Option<u64>,
) -> (i128, i128) {
    recipient.require_auth();
    assert!(share_amount > 0, "withdraw_liquidity: zero share_amount");
    let reserve_a: i128 = e
    ... (truncated)
```

Recommendation:

Burn (or transfer to pool) the caller's share tokens and/or verify `recipient`'s share balance before computing withdrawal amounts.

● LOW Anyone can mint unlimited mock tokens

Description:

`MockToken::mint` has no authorization check and ignores the stored `Admin`. Any caller can mint arbitrary amounts to any address, breaking any integration-test assumption that token supply is controlled. If this mock were ever deployed/used beyond tests, an attacker could mint unbounded balance and interact with downstream vault/strategy logic as if they had real funds.

Affected Code:

```
pub fn initialize(env: Env, admin: Address) {
    assert!(
        !env.storage().instance().has(&TKey::Admin),
        "already initialized"
    );
    env.storage().instance().set(&TKey::Admin, &admin);
}

pub fn mint(env: Env, to: Address, amount: i128) {
    assert!(amount >= 0, "mint: negative
... (truncated)
```

Recommendation:

If this contract can be invoked outside a fully-controlled test harness, gate `mint` with `admin.require\_auth()` (and consider also requiring auth in `initialize` so an attacker can't front-run initialization).

● LOW Phoenix pool can be reinitialized by anyone

Description:

'MockPhoenixPool::phoenix_init' can be called repeatedly and has no 'require_auth' or "already initialized" guard. Any caller can reset the pool's share token and underlying token addresses and zero/overwrite reserves, breaking invariants assumed by integrations. If this mock were used in a less controlled environment, an attacker could swap in attacker-controlled token contracts or reset reserves to affect subsequent liquidity/withdraw computations.

Affected Code:

```
pub fn phoenix_init(env: Env, share_token: Address, token_a: Address, token_b: Address) {
    env.storage()
        .instance()
        .set(&PhoenixKey::ShareToken, &share_token);
    env.storage()
        .instance()
        .set(&PhoenixKey::UnderlyingA, &token_a);
    env.storage()
        .inst
    ... (truncated)
```

Recommendation:

Add a one-time initialization guard (e.g., check a stored init flag) and restrict initialization to an admin ('require_auth').

contracts/vault/src/lib.rs

● HIGH Vault can be reinitialized to seize control

Description:

The contract's `__constructor` performs full initialization (setting manager/trader/base_asset/share_token/fees, clearing announced fees, etc.) but does not include an initialization guard (e.g., `if is_initialized { revert }`). In Soroban, `__constructor` is an exported contract function name and—absent an explicit guard—can be invoked again after deployment. An attacker can call `__constructor` with `params.manager=params.trader=attacker` (and authorize both), overwriting the stored manager/trader and other critical configuration. This enables subsequent privileged actions (setting strategies/guards/oracle, pausing, fee changes, investing/trading) and can permanently break accounting by changing base/share token addresses. The share-token admin takeover step is also executed during `__constructor`. If the share token's current admin is not already the vault, the attacker can set `params.share_token` to a token they control and make the vault set itself as admin, giving the attacker indirect mint/burn control via the now-compromised vault roles. Exploit path: 1) After deployment, attacker submits a tx calling `Vault::__constructor` with attacker-controlled addresses in `VaultParams` and signs for both `manager` and `trader`. 2) Vault overwrites `Manager`/`Trader`/`BaseAsset`/`ShareToken` and fee parameters. 3) Attacker (now manager/trader) uses `invest`/`unwind`/`execute_trade`/fee setters to redirect funds, whitelist malicious strategies, or freeze operations. This is a protocol-takeover class issue if `__constructor` is callable post-deploy (common in Soroban unless guarded).

Affected Code:

```
pub fn __constructor(env: Env, params: VaultParams) {
    params.manager.require_auth();
    params.trader.require_auth();
    ...
    set_manager(&env, &params.manager);
    set_trader(&env, &params.trader);
    set_base_asset(&env, &params.base_asset);
    set_share_token(&env, &params.share_token
... (truncated)
```

Recommendation:

Add a one-time initialization guard at the start of `__constructor`, e.g.:

- `if storage::is_initialized(&env) { panic_with_error!(&env, VaultError::AlreadyInitialized) }`
- set an explicit `Initialized` flag in instance storage.

Also consider renaming to `initialize` with explicit guard semantics, ... (truncated)

● MEDIUM Unchecked i128 multiplication can wrap share math

Description:

Several core accounting computations multiply `i128` values using `\*` without checked/saturating arithmetic. In release WASM builds, Rust integer overflow wraps, which can produce incorrect share prices and mint/burn amounts. Examples: - Deposit share calculation: `amount \* PRICE\_PRECISION / share\_price`. - Share price calculation: `nav \* PRICE\_PRECISION / total\_supply`. - Withdraw base gross: `share\_amount \* share\_price / PRICE\_PRECISION`. If `nav`, `amount`, or `share\_amount` become large (e.g., high-supply tokens, misconfigured decimals, or a malicious token/strategy/oracle reporting extreme values), these multiplications can overflow and wrap, leading to severe mis-accounting (potentially minting an outsized number of shares or allowing underpriced withdrawals). Exploitability depends on whether an attacker can cause large intermediate values (e.g., via reinitialization, malicious base_asset/oracle, or unusually large token supplies).

Affected Code:

```
let total_shares = if share_price == 0 || total_supply == 0 { amount } else { amount * PRICE_PRECISION / share_price };
...
fn share_price(nav: i128, total_supply: i128) -> i128 {
    if total_supply == 0 || nav == 0 { PRICE_PRECISION } else { nav * PRICE_PRECISION / total_supply }
}
...
let base_gr
... (truncated)
```

Recommendation:

Use `checked\_mul`/`checked\_div` (or `saturating\_\*` with explicit error handling) for all NAV/share-price/share-mint calculations, and revert on overflow. Consider reordering operations to divide earlier when safe (while preserving precision), and add upper-bound sanity checks on NAV and supplies.

● LOW Deposits and withdrawals lack user slippage bounds

Description:

`deposit` and `withdraw` execute at the vault's current NAV/share price without any caller-provided minimum/maximum bounds: - `deposit` has no `min\_shares\_out`. - `withdraw` has no `min\_base\_out`. A user who signs a transaction based on an observed share price can be adversely affected if NAV changes before their transaction executes (e.g., manager executes a large trade/invest/unwind in the same ledger or just before inclusion; strategy/oracle values shift). This is a classic stale-state / sandwich risk: the user can receive fewer shares than expected on deposit or less base than expected on withdrawal, with no on-chain protection. While the manager is privileged, third parties can also influence NAV indirectly if any whitelisted strategy's `get\_value` depends on manipulable on-chain pool reserves or oracle prices (especially when `ValueManipulationGuardEnabled` is left disabled by default).

Affected Code:

```
pub fn deposit(env: Env, amount: i128, from: Address) -> i128 {
    ...
    let share_price = Self::share_price(nav, total_supply);
    let total_shares = ... amount * PRICE_PRECISION / share_price;
    ...
    share::mint(&env, &share_token, &from, user_shares);
    ...
}

pub fn withdraw(env: Env,
    ... (truncated)
```

Recommendation:

Add user-provided bounds:

- `deposit(amount, from, min\_shares\_out)` and revert if `user\_shares < min\_shares\_out`.
- `withdraw(share\_amount, from, to, min\_base\_out)` and revert if `base\_net < min\_base\_out`.

Optionally expose `preview\_deposit`/`preview\_withdraw` view helpers so clients can compute bou...
(truncated)

contracts/share_token/src/lib.rs

● CRITICAL Callable constructor allows admin takeover reinit

Description:

`\_\_constructor` writes all instance state (admin/name/symbol/decimals/total_supply) but never checks whether the contract is already initialized. In Soroban, `\_\_constructor` is still an exported entrypoint and can be invoked after deployment like any other function. Because it only requires auth from the *passed-in* `admin` parameter (not the currently-stored admin), any attacker can call `\_\_constructor` again, sign as their own address, and overwrite the stored admin. Exploitation path: 1) Contract is deployed and initialized normally. 2) Attacker calls `\_\_constructor(env, attacker\_addr, ...)` and signs as `attacker\_addr`. 3) Storage is overwritten so `Admin = attacker\_addr` and `TotalSupply = 0`. 4) Attacker calls `mint()` (now authorized) to mint arbitrary shares to themselves, breaking vault/share accounting and potentially enabling theft wherever these shares are accepted/redeemable.

Affected Code:

```
pub fn __constructor(env: Env, admin: Address, name: String, symbol: String, decimals: u32) {
    admin.require_auth();

    set_admin(&env, &admin);
    set_name(&env, &name);
    set_symbol(&env, &symbol);
    set_decimals(&env, decimals);
    set_total_supply(&env, 0_i128);

    env.storage()

    ... (truncated)
```

Recommendation:

Add a one-time initialization guard at the start of `\_\_constructor`, e.g. `if storage::has\_admin(&env) { panic\_with\_error!(&env, ShareTokenError::AlreadyInitialized) }`, and only then set instance fields. Consider also making all getters consistently enforce initialization (including `total\_supply`)... (truncated)

● LOW `total_supply` succeeds even when uninitialized

Description:

`total_supply()` is documented to error when the contract is not initialized, but `get_total_supply()` defaults to `0` when the value is missing. If integrators use `total_supply()==0` as a valid initialized-state signal, this can cause incorrect behavior (e.g., proceeding with logic against an uninitialized token contract).

Affected Code:

```
pub fn total_supply(env: Env) -> i128 {
    env.storage()
        .instance()
        .extend_ttl(INSTANCE_LIFETIME_THRESHOLD, INSTANCE_BUMP_AMOUNT);
    get_total_supply(&env)
}

pub fn get_total_supply(env: &Env) -> i128 {
    env.storage()
        .instance()
        .get(&DataKey::TotalSupply)

    ... (truncated)
```

Recommendation:

Make `get_total_supply()` consistent with other instance getters by panicking with `NotInitialized` when `TotalSupply` is absent, or explicitly document that `total_supply()` returns `0` pre-init and ensure callers do not treat that as initialized state.

contracts/oracle/src/storage.rs

● MEDIUM Instance keys expiry disables oracle staleness checks

Description:

`MaxAgeLedgers` is stored in instance storage, but `get\_max\_age\_ledgers()` defaults to `0` when the key is missing. In the Oracle contract, `max\_age == 0` disables staleness checks. If the contract is inactive long enough for **instance storage to expire** (short TTL) while **persistent `Price` entries still exist** (long TTL), then subsequent `get\_price` calls will treat `max\_age` as `0` and return prices without any freshness enforcement. This can enable stale-price usage/arbitrage in downstream contracts (e.g., vault share pricing) after a period of inactivity: an attacker waits until instance state expires, then calls `get\_price` (which also extends the persistent TTL via `get\_price\_data`) to keep old prices alive and usable indefinitely, while freshness checks remain disabled.

Affected Code:

```
pub fn get_max_age_ledgers(env: &Env) -> u32 {
    env.storage()
        .instance()
        .get(&DataKey::MaxAgeLedgers)
        .unwrap_or(0)
}
```

Recommendation:

Do not treat a missing `MaxAgeLedgers` as `0`. Instead: (a) revert with `NotInitialized` if `MaxAgeLedgers` is absent, or (b) return a safe default (e.g., `DEFAULT\_MAX\_AGE\_LEDGERS`) and/or store `MaxAgeLedgers` in persistent storage with the same TTL regime as `Price`. Consider additionally assertin... (truncated)

● MEDIUM Admin stored in shorter-lived Instance storage

Description:

The oracle `Admin` is stored in instance storage (short TTL), while prices are stored in persistent storage (long TTL). After sufficient inactivity, the admin key can expire while price entries remain. This can leave the oracle permanently unmaintainable (no admin => cannot update prices/max-age), while still serving old prices from persistent storage. Combined with the `get\_max\_age\_ledgers()` missing-key defaulting behavior, this can result in an oracle that continues returning stale prices without freshness checks and without any way to remediate via `set\_price`.

Affected Code:

```
pub enum DataKey {  
    /// Admin address – instance storage.  
    Admin,  
    /// Maximum acceptable age for price data in ledgers.  
    MaxAgeLedgers,  
    /// Price of an asset, PRICE_PRECISION-scaled – persistent storage.  
    Price(Address),  
}  
  
pub fn set_admin(env: &Env, admin: &Address) {  
    env.s  
    ... (truncated)
```

Recommendation:

Store `Admin` (and other critical config like `MaxAgeLedgers`) in persistent storage, or ensure instance TTL is always kept $\geq$ persistent TTL, and ensure read paths fail closed when initialization/config is missing.

contracts/strategies/blend/src/lib.rs

● MEDIUM Short instance TTL risks state expiry DoS

Description:

All entrypoints extend `*instance*` storage TTL using ``INSTANCE_LIFETIME_THRESHOLD`/`INSTANCE_BUMP_AMOUNT`` (≈1.2/2.4 days per comments). If the strategy is idle for a few days, its instance state (vault/asset/protocol/manager) can expire. After expiry, ``get_vault/get_asset/get_protocol/get_manager`` will revert as ``NotInitialized``, preventing deposits/withdrawals and potentially blocking users from exiting positions until the authorized parties re-initialize (which may be operationally difficult during incidents).

Affected Code:

```
env.storage()
  .instance()
  .extend_ttl(INSTANCE_LIFETIME_THRESHOLD, INSTANCE_BUMP_AMOUNT);
```

Recommendation:

Use persistent storage for critical configuration, or increase TTL bump/threshold to a realistic operational window (weeks+), and/or add a permissionless “bump TTL” function to keep the instance alive without requiring manager actions.

● LOW Protocol allowance not revoked after supply

Description:

``deposit`` approves the configured ``protocol`` contract to spend ``amount`` of the strategy's ``asset``, but never revokes the allowance after ``blend_submit``. If the ``protocol`` address is (now or later) malicious/compromised (or exposes any method that can trigger ``token.transfer_from`` using its own spender auth), it can drain any ``asset`` tokens sitting in the strategy contract up to the remaining allowance until expiry. This is especially relevant if (a) the pool doesn't fully consume the allowance, or (b) tokens are later transferred directly to the strategy address (accidentally or via dusting).

Affected Code:

```
// Let the lending protocol pull supplied funds from this strategy.
let expiry = env.ledger().sequence() + 100;
token::Client::new(&env, &asset).approve(&strategy_addr, &protocol, &amount, &expiry);

// Supply to Blend; strategy is the account holder.
let requests: Vec<BlendRequest> = soroban_sdk::v
... (truncated)
```

Recommendation:

Revoke/reset the allowance to ``0`` immediately after ``blend_submit`` (similar to how the vault revokes strategy allowance). Alternatively, prefer a call pattern where the pool pulls exactly what it needs without leaving residual allowance, if supported.

contracts/strategies/soroswap_lp/src/lib.rs

● MEDIUM LP valuation uses saturating math masking overflow

Description:

`get\_value` uses `saturating\_mul`/`saturating\_add` for reserve × price computations. If `reserve\_ \* price\_` or subsequent sums overflow `i128`, the result silently saturates to `i128::MAX` instead of reverting. Vault NAV and share-price calculations call `strategy.get\_value(vault)`; a saturated return value can greatly overstate NAV, potentially enabling share mispricing (e.g., minting too many shares on deposit or returning too much base on withdraw) depending on how the vault uses the value. This can be triggered if pool reserves and/or oracle prices are large enough (or if oracle precision is misconfigured / malicious), even when all values are positive.

Affected Code:

```
let pool_value_a = reserve_a.saturating_mul(price_a) / PRICE_PRECISION;
let pool_value_b = reserve_b.saturating_mul(price_b) / PRICE_PRECISION;
let pool_value = pool_value_a.saturating_add(pool_value_b);

pool_value.saturating_mul(lp_balance) / total_lp
```

Recommendation:

Use checked arithmetic and revert on overflow (e.g., `checked\_mul`/`checked\_add` with explicit error) rather than saturating, so valuation cannot silently jump to extreme values. Consider bounding oracle prices and reserves to expected ranges/decimals before multiplication.

● LOW Router return values can overflow dust accounting

Description:

After `router.add_liquidity`, the contract computes `dust_a = amount_a - a_used` and `dust_b = amount_b - b_used` using unchecked subtraction. If a malicious/misconfigured router returns `a_used`/`b_used` outside the expected range (e.g., greater than the input amounts or extreme values), subtraction can overflow/wrap and either revert unexpectedly or compute an incorrect `dust_*` amount. This can brick deposits (DoS) or attempt incorrect dust transfers. While the router is intended to be trusted, this strategy stores `router` from initialization and does not enforce that it is a genuine Soroswap router contract.

Affected Code:

```
let dust_a = amount_a - a_used;
let dust_b = amount_b - b_used;
if dust_a > 0 {
    token::Client::new(&env, &asset_a).transfer(&strategy, &from, &dust_a);
}
if dust_b > 0 {
    token::Client::new(&env, &asset_b).transfer(&strategy, &from, &dust_b);
}
```

Recommendation:

Validate router outputs before arithmetic (e.g., require `0 <= a_used && a_used <= amount_a` and same for `b_used`), and use `checked_sub` with a clear error to prevent overflow/wrap.

contracts/strategies/phoenix_lp/src/lib.rs

● MEDIUM Unchecked oracle prices can inflate NAV to max

Description:

`get\_value` computes LP position value from pool reserves and oracle prices but performs no sanity checks that oracle prices are positive and within a reasonable range. Because the arithmetic uses `saturating\_mul`/`saturating\_add`, an abnormally large oracle price can saturate intermediate results to `i128::MAX`, causing the strategy to report an extremely large NAV. This strategy is used by the vault to compute NAV for LP strategies (vault treats LP `get\_value()` as already in base-asset units). If the configured oracle can be manipulated (e.g., weak on-chain oracle, compromised oracle contract, or stale/incorrect price feed), an attacker could inflate the reported NAV and then exploit vault share minting/redemption logic that depends on NAV (e.g., minting vault shares with comparatively little real value, diluting other depositors). While the oracle address itself is manager-controlled, price-manipulation risk is a common and realistic attack vector if the oracle is not strongly secured and bounded.

Affected Code:

```
let price_a = oracle_client.get_price(&asset_a);
let price_b = oracle_client.get_price(&asset_b);

let pool_value_a = reserve_a.saturating_mul(price_a) / PRICE_PRECISION;
let pool_value_b = reserve_b.saturating_mul(price_b) / PRICE_PRECISION;
let pool_value = pool_value_a.saturating_add(pool_value_b
... (truncated)
```

Recommendation:

Add defensive validation in `get\_value`: reject non-positive prices and cap prices/reserves to expected maximums (or use checked math and revert on overflow instead of saturating). Consider incorporating oracle staleness/validity checks (timestamp/sequence) and/or using a robust TWAP oracle. If satu...
(truncated)

● LOW Internal share tracking can desync from real balance

Description:

The strategy maintains its own `TotalShares` counter and uses it as the source of truth for `get\_share\_balance` and for limiting withdrawals. However, this value can diverge from the actual SEP-41 share-token balance held by the strategy contract. Examples that cause desynchronization include: - External transfers of LP share tokens to the strategy (airdrop, mistaken transfer), increasing real balance without increasing `TotalShares`. - A non-standard/malicious share token that can claw back or burn shares from the strategy, decreasing real balance without decreasing `TotalShares`. If `TotalShares` is lower than the real balance, some value can become permanently unwithdrawable through this strategy because `withdraw` blocks amounts greater than `TotalShares`. If `TotalShares` is higher than the real balance, withdrawals will revert inside the pool/token transfer path (DoS) until governance intervenes. This is especially relevant because the vault relies on `get\_total\_shares` for reporting (`get\_share\_balance`) and valuation (`get\_value` uses `shares = get\_total\_shares`) rather than querying the actual share-token balance.

Affected Code:

```
set_total_shares(&env, 0);
...
let shares_before = token::Client::new(&env, &share_token).balance(&strategy);
...
let shares_after = token::Client::new(&env, &share_token).balance(&strategy);
...
let new_total = get_total_shares(&env)
    .checked_add(shares_minted)
    .unwrap_or_else(|| panic_with
... (truncated)
```

Recommendation:

Prefer using the actual share-token balance (`token::Client(balance)`) as the source of truth, or regularly reconcile `TotalShares` against the real balance (e.g., before/after withdrawals, or via an admin-only `sync\_shares()` function). If `TotalShares` must be kept for gas/consistency, add asserti... (truncated)

contracts/integration_tests/src/test_factory_registry.rs

● LOW Auth checks not validated due to mock_all_auths

Description:

The test setup calls `env.mock_all_auths()`, which makes all `require_auth` checks succeed. As a result, these integration tests cannot detect missing/mis-scoped `require_auth` in the Factory/Vault contracts. The “non-admin should panic” tests will still pass if the contract only enforces admin via a parameter check (or even if `require_auth` is accidentally removed elsewhere), creating a false sense of access-control coverage.

Affected Code:

```
fn setup() -> World {
    let env = Env::default();
    env.mock_all_auths();

    let admin = Address::generate(&env);
    let factory_id = env.register(Factory, (admin.clone(),));
    let factory = FactoryClient::new(&env, &factory_id);

    let factory: FactoryClient<'static> = unsafe { core::mem
... (truncated)
```

Recommendation:

For at least one test suite that is meant to validate authorization, avoid `mock_all_auths()` and instead use explicit auth setup (e.g., `mock_auths`/recorded auth assertions) to ensure privileged entrypoints actually call `require_auth` on the correct address(es).

contracts/trade_guards/phoenix/src/lib.rs

● HIGH Slippage check compares different-asset amounts

Description:

`validate\_swap`/`validate\_swap\_exact\_in` call `check\_slippage(min\_out, amount\_in, ...)` effectively enforcing `(amount\_in - min\_out)/amount\_in <= MAX\_SLIPPAGE\_BPS`. However, `min\_out` is denominated in the *final output token*, while `amount\_in` is denominated in the *input token*; comparing them is not a meaningful slippage bound unless the two assets have identical units/price. In addition, the vault codebase comments indicate guards should fetch an on-chain quote internally and not rely on a caller-supplied quote, but this Phoenix guard does not fetch any quote at all. As a result, a malicious/compromised vault trader can set `min\_out` to any value that passes this unit-mismatched check (often trivially satisfied when `amount\_in` is large in its token units), allowing swaps with effectively unbounded price impact as long as the tokens are whitelisted.

Affected Code:

```
Self::check_slippage(min_out, amount_in, &env);

fn check_slippage(min_out: i128, quoted_out: i128, env: &Env) {
    if min_out < 0 || quoted_out <= 0 || min_out > quoted_out {
        panic_with_error!(env, PhoenixGuardError::SlippageTooHigh);
    }
    let diff = quoted_out - min_out;
    let lhs
    ... (truncated)
```

Recommendation:

Compute slippage using values in the same units: either (a) fetch a trusted quote in output-token units (e.g., via the strategy/router wrapper's `quote\_exact\_in`) and compare `min\_out` against that quote, or (b) require the vault to pass a quote and bind it with `require\_auth\_for\_args` to prevent sp... (truncated)

contracts/vault/src/storage.rs

● MEDIUM Temporary TTL extension likely ineffective with threshold 0

Description:

`set\_last\_deposit\_ts` and `set\_op\_state` attempt to bound per-user temporary storage lifetime via `extend\_ttl(&key, 0, ttl)`. In Soroban, `extend\_ttl` is conditional on the remaining TTL being below a threshold; using `threshold = 0` commonly results in *no extension* unless the entry is at/near expiration. If so, the cooldown timestamp may expire at the network's default temporary-entry TTL instead of `cooldown\_ledgers + INSTANCE\_LIFETIME\_THRESHOLD`, allowing users to bypass long configured cooldowns once the temp entry expires early. Similarly, `OpState` may persist for an unintended duration or not long enough for same-ledger checks, weakening or misbehaving the value-manipulation guard. Attack path (cooldown bypass): manager sets a cooldown longer than the network's default temporary TTL; a depositor waits until their `LastDepositTs` temp entry expires (earlier than configured) then withdraws without satisfying the intended cooldown. Because this hinges on Soroban SDK/host TTL semantics, confirm by testing `extend\_ttl(&key, 0, ...)` behavior on your target protocol version; if `threshold=0` is a no-op, this is exploitable as described.

Affected Code:

```
pub fn set_last_deposit_ts(env: &Env, user: &Address, ts: u64) {
    let key = DataKey::LastDepositTs(user.clone());
    env.storage().temporary().set(&key, &ts);

    let cooldown_secs = get_exit_cooldown_secs(env);
    let cooldown_ledgers = cooldown_secs
        .saturating_add(SECS_PER_LEDGER -
... (truncated)
```

Recommendation:

Use a threshold that guarantees extension occurs when you set the key (e.g., a large threshold such as `u32::MAX` or at least `ttl / INSTANCE\_LIFETIME\_THRESHOLD`, per the host's exact condition), and ensure `extend\_to` respects `env.storage().max\_ttl()`. Add an integration test that sets a long cool... (truncated)

contracts/trade_guards/soroswap/src/lib.rs

● HIGH Multiple one-time initializers can be front-run and permanently hijacked

Description:

Several contract instances expose a permissionless one-time `initialize` entrypoint that only checks `is\_initialized` and then accepts a caller-supplied `vault`. Because authorization is taken from `vault.get\_manager()` on the provided vault rather than from a predetermined deployer or expected vault instance, an attacker can deploy or supply a malicious vault they control, satisfy the auth checks, and initialize the contract first. This can permanently set vault/manager/asset/strategy/whitelist state to attacker-chosen values and prevent the intended deployment from initializing later (DoS/configuration hijack).

Affected Code:

```
pub fn initialize(...) {
    if is_initialized(&env) {
        panic_with_error!(&env, ...::AlreadyInitialized);
    }

    let vault_manager: Address = env.invoke_contract(
        &vault,
        &Symbol::new(&env, "get_manager"),
        ().into_val(&env),
    );
    vault_manager.require_auth();
    ... (truncated)
```

Recommendation:

Make initialization atomic at deployment time (e.g., use `\_\_constructor` or factory deploy+init in the same transaction). If a separate initializer must remain, restrict it to a trusted deployer/admin, bind it to a known expected vault instance, and/or require auth from the actual vault or another f... (truncated)